

Custom Web Scraper

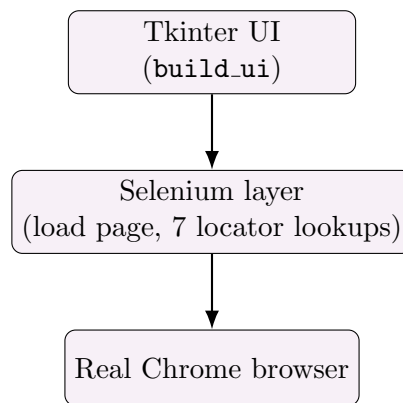
Brief: What It Does and How

Explainer document generated for the project owner

1 One-Line Summary

A Tkinter desktop app that drives a real Chrome browser (via Selenium) to a URL you type, then finds and highlights elements on that page using one of seven locator strategies (class, ID, name, tag, link text, partial link text, XPath), showing matches in a results table.

2 Architecture



One file, one live Chrome session, no database. A third, pure formatting layer (`summarize_element`, `build_result_rows`) turns raw Selenium `WebElement` objects into plain tuples for the results table and can be unit-tested without a browser.

3 Core Flow

1. **Load a page.** `driver.get(url)` navigates Chrome, then `WebDriverWait(driver, 10)` waits for `<html>` and `<body>` to appear before declaring the page loaded, catching a bad URL (`InvalidArgumentException`) or a slow/dead page (`TimeoutException`) with a specific status message either way.
2. **Pick a locator and search.** A dispatch dictionary (`LOCATOR_DISPATCH`) maps the selected strategy name to one of seven near-identical `get_element_by_*` functions. Each waits up to 10 seconds for a match, then calls `driver.find_elements(...)`; a locator that never matches times out rather than raising a parse error, which is why every lookup is wrapped in `WebDriverWait/TimeoutException`.
3. **View results.** Matches are summarized (tag, truncated text, key attributes) and shown in a `ttk.Treeview` table in a new window.

4. **Highlight.** A “Highlight in Browser” button runs a small `driver.execute_script(...)` JavaScript snippet against each matched element to outline it and scroll it into view in the live Chrome window.

4 Notable Design Choices

- **Shared globals, not a class.** `driver`, `is_request_loaded`, and `last_elements` are module-level variables read directly by every function. Simple for a single-window, single-browser-session script; the cost is that the seven locator functions cannot be unit-tested without a live driver and window.
- **Selenium Manager over a hardcoded path.** An earlier version pointed at a fixed Windows path to `chromedriver.exe`. This version resolves the driver automatically (or via an optional `CHROMEDRIVER_PATH` override), so the same code runs on Linux, macOS, or Windows.
- **“No page loaded” guard.** Every locator function checks `is_request_loaded` first, so clicking “Get Elements” before “Request” shows a clear message instead of throwing.

5 From an Earlier Backend

The project’s first version used `requests` + BeautifulSoup with CSS selectors, which cannot see JavaScript-rendered content and only raised a plain parse error on a bad selector. Switching to Selenium fixed the JavaScript gap but introduced a new failure mode, a bad locator times out instead of raising, which the current code handles explicitly with `WebDriverWait` and `TimeoutException`.

Honest limitation. This app needs a real display and outbound network access for a full end-to-end run (Selenium Manager downloads a matching `chromedriver` on first use). The environment this project was verified in has no outbound network access, so an actual live-site scrape and the “Highlight in Browser” JavaScript were never exercised end to end here. What was verified: the file compiles; the pure formatting functions pass a unit test suite against fake objects; and the full Tkinter window (including the highlight button’s enabled state) was built and driven against a fake driver on a live display. The real Selenium calls used are standard, stable 4.x API, but that is not a substitute for a live run.

6 Running It

```
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python main.py
```

Needs Python 3.9+, Chrome/Chromium, and (`python3-tk` on Debian/Ubuntu if not already installed).